

Universidad de San Carlos de Guatemala.
Centro Universitario de Occidente.
División de Ciencias de la Ingeniería.
Organización de Lenguajes y Compiladores II.
Ing. Mario Tobar.



USAC
TRICENTENARIA
Universidad de San Carlos de Guatemala

MANUAL DE USUARIO

Estudiante	Carné
201931811	Diego José Maldonado Monterroso

Quetzaltenango, Guatemala.

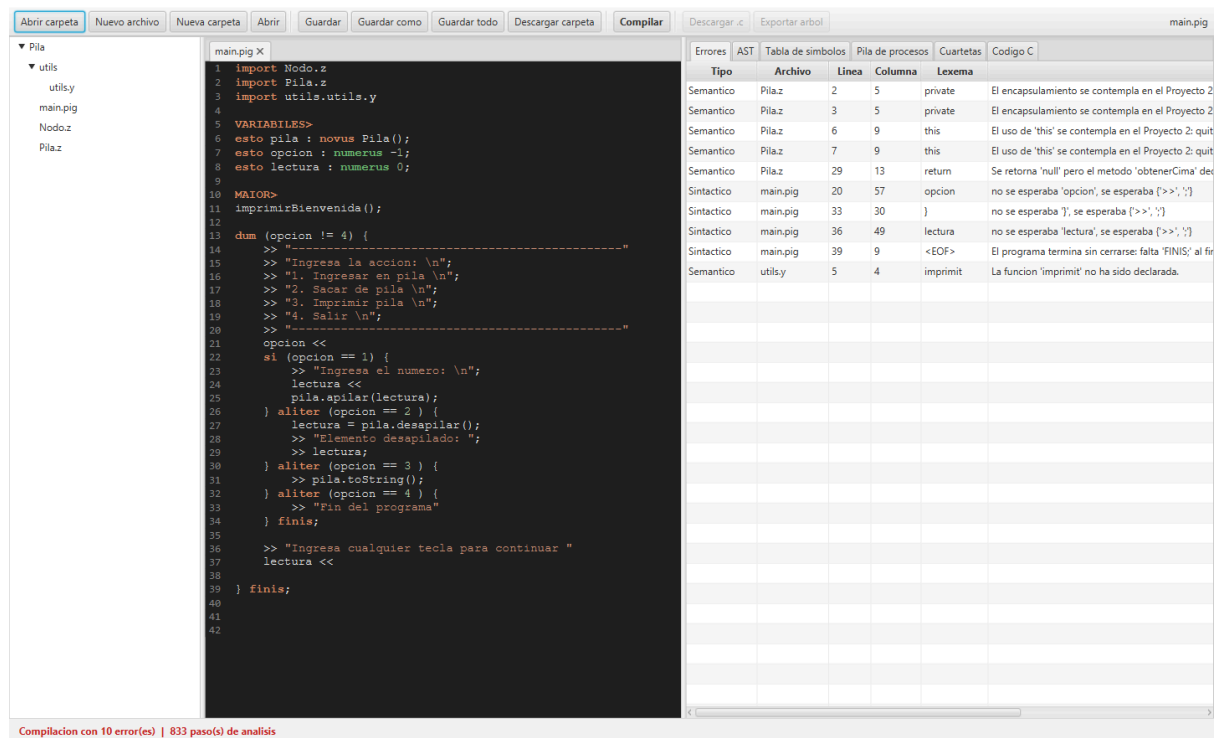
Sábado 26 de septiembre de 2026.

REQUISITOS Y ARRANQUE

- JDK 21 o superior y Maven 3.9.
- Para ejecutar el C generado: `gcc` (en Windows, dentro de WSL).

```
cd contacto_3xtrat3rr3str3D  
mvn clean javafx:run
```

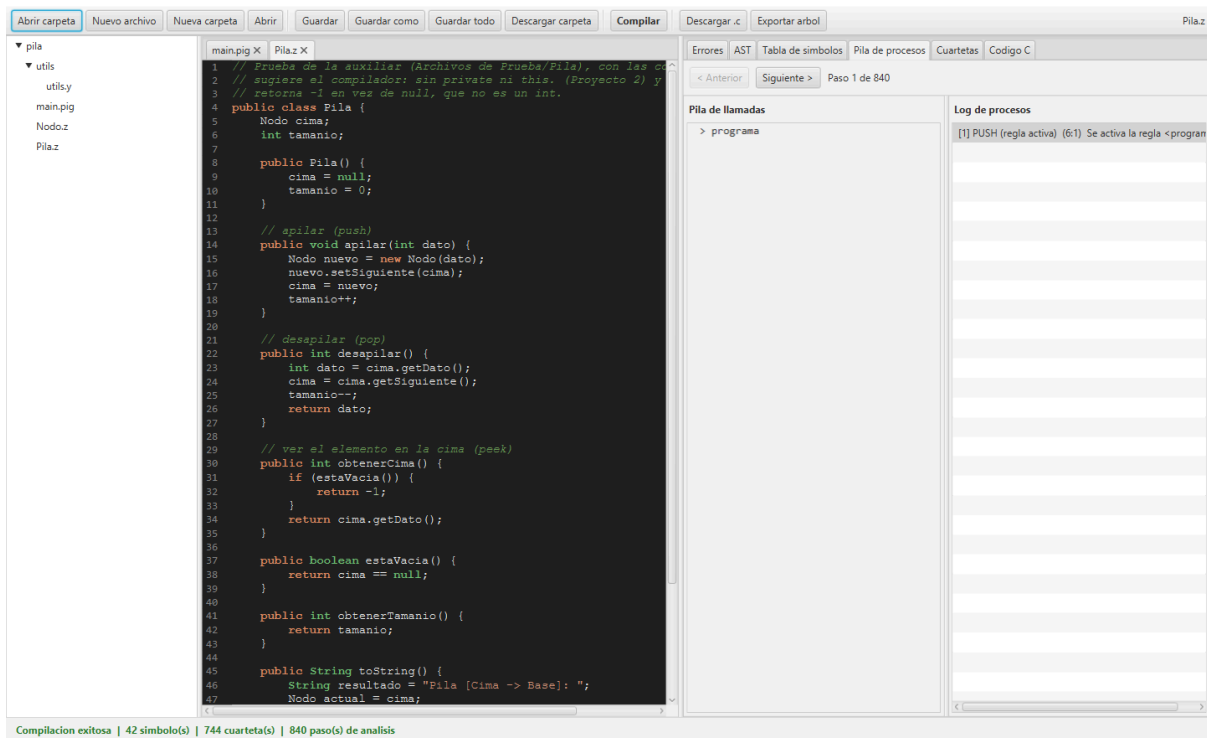
VISTAS



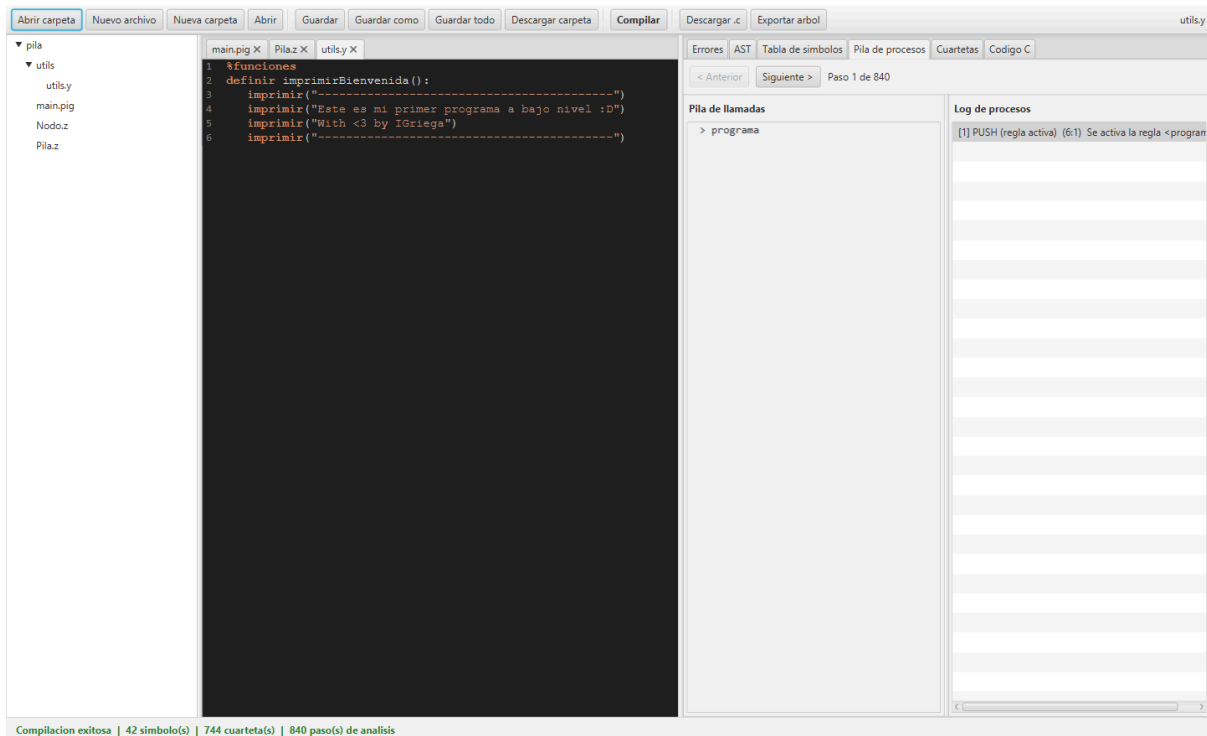
Ventana principal

Zona	Qué hace
Barra superior	archivos y carpetas, Compilar , Descargar .c , Exportar arbol (se habilita tras una compilación sin errores; Descargar .c , además, solo con un .pig)
Izquierda	árbol de la carpeta de trabajo (doble clic abre un archivo)
Centro	una pestaña por archivo, coloreada según su lenguaje; * = cambios sin guardar
Derecha	Errores , AST , Tabla de simbolos , Pila de procesos , Cuartetas , Codigo C
Barra inferior	resultado de la última compilación

El editor colorea en tiempo real, mientras se escribe, con el analizador léxico del propio compilador: palabras reservadas, tipos, literales, comentarios y operadores de cada lenguaje según la extensión del archivo.



Coloreado de un archivo Zetariano (.z)



Coloreado de un archivo Y? (.y)

ARCHIVOS Y CARPETAS

Botón	Acción
Abrir carpeta	elige la carpeta de trabajo y la muestra en el árbol
Nuevo archivo	crea un archivo dentro de la carpeta seleccionada en el árbol (sin carpeta abierta: pestaña sin título)
Nueva carpeta	crea una subcarpeta en la carpeta seleccionada
Abrir	abre un .pig, .y o .z suelto y lo compila
Guardar / Guardar como	guarda la pestaña activa; <i>Guardar como</i> también sirve para descargar el archivo en otra ubicación
Guardar todo	guarda todas las pestañas con cambios
Descargar carpeta	copia la carpeta de trabajo completa a donde elija

Al cerrar una pestaña, o la ventana, con cambios sin guardar, se pide confirmación.

COMPILAR

1. Abra la carpeta del proyecto y el `.pig` principal (el que tiene `MAIOR>`).
2. Pulse **Compilar**. Se compila la pestaña activa; los archivos abiertos con cambios se guardan antes, porque los `import` se leen del disco.
3. Los `import` se resuelven desde la carpeta del `.pig`: `import utils.utils.y` es `utils/utils.y`. Distinguen mayúsculas también en Windows: `import Utils.utils.y` es un error si la carpeta se llama `utils`.

Un `.y` o `.z` también se puede compilar solo, para revisar sus errores; solo un `.pig` genera cuartetos y C.

LEER ERRORES

La pestaña **Errores** lista tipo (léxico, sintáctico, semántico), archivo, línea, columna, lexema y descripción. **Doble clic** en un error lleva el cursor a su posición, **aunque esté en un archivo importado**: lo abre.

The screenshot shows an IDE with a project named 'Pila'. The main editor displays the code for 'Pila.java'. The right-hand pane shows a list of errors:

Tipo	Archivo	Línea	Columna	Lexema	Descripción
Semántico	Pila.z	2	5	private	El encapsulamiento se contempla en el Proyecto 2
Semántico	Pila.z	3	5	private	El encapsulamiento se contempla en el Proyecto 2
Semántico	Pila.z	6	9	this	El uso de 'this' se contempla en el Proyecto 2: quite
Semántico	Pila.z	7	9	this	El uso de 'this' se contempla en el Proyecto 2: quite
Semántico	Pila.z	29	13	return	Se retorna 'null' pero el metodo 'obtenerCima' declara
Sintactico	main.pig	20	57	opcion	no se esperaba 'opcion', se esperaba { '>>', ';' }
Sintactico	main.pig	33	30	}	no se esperaba '}', se esperaba { '>>', ';' }
Sintactico	main.pig	36	49	lectura	no se esperaba 'lectura', se esperaba { '>>', ';' }
Sintactico	main.pig	39	9	<EOF>	El programa termina sin cerrarse: falta 'FINIS' al fir
Semántico	utils.y	5	4	imprimir	La funcion 'imprimir' no ha sido declarada.

Compilación con 10 error(es) | 833 paso(s) de análisis

Salto a un error dentro de Pila.z

Los mensajes sugieren la corrección. Con la prueba `Pila` de la auxiliar:

Mensaje	Corrección
El encapsulamiento se contempla en el Proyecto 2: quite 'private' de 'cima'.	borrar <code>private</code>
El uso de 'this' se contempla en el Proyecto 2: quite 'this.' ...	<code>this.cima = null;</code> → <code>cima = null;</code>
Se retorna 'null' pero el metodo 'obtenerCima' declara 'int'.	retornar un entero, p. ej. <code>return -1;</code>
no se esperaba 'opcion', se esperaba { '>>', ';' }	agregar <code>;</code> al final de la línea anterior
El programa termina sin cerrarse: falta 'FINIS;' al final.	agregar <code>FINIS;</code>

Mensaje	Corrección
<i>La funcion 'imprimit' no ha sido declarada.</i>	corregir el nombre: <code>imprimir</code>

Con errores no se grafica el AST ni se genera código.

RESULTADOS DE COMPILACIÓN EXITOSA

Pestaña	Contenido
Código C	el programa en C; Descargar .c lo guarda junto al .pig con el mismo nombre
Cuartetas	una fila por instrucción: operador, arg1, arg2, resultado
AST	el árbol del programa con la línea:columna de cada nodo; doble clic salta al código; Exportar arbol lo guarda como texto
Tabla de simbolos	variables, arreglos, estructuras, clases, atributos, funciones y métodos, con su ámbito y su celda del stack
Pila de procesos	los pasos del analizador sintáctico, con <i>Anterior / Siguiente</i> ; también se llena cuando hay errores

The screenshot shows a code editor with two panes. The left pane displays the C source code for a stack implementation, and the right pane shows the compilation output.

Left Pane (main.pig X):

```

1 // Prueba de la auxiliar con las correcciones que sugiere el...
2 // los ";" faltantes (lineas 20, 33 y 36) y el "FINIS;" final...
3 //
4 // Entrada de ejemplo, un valor por línea: 1 5 x 1 7 x 3 x 2 a
5 // Imprime la pila "7, 5", desapila 7 y luego imprime "5".
6 import Nodo.z
7 import Pila.z
8 import utils.utils.y
9
10 VARIABLES>
11 esto pila : novus Pila();
12 esto opcion : numerus -1;
13 esto lectura : numerus 0;
14
15 MAIOR>
16 imprimirBienvenida();
17
18 dum (opcion != 4) {
19     >> "-----"
20     >> "Ingresa la accion: \n";
21     >> "1. Ingresar en pila \n";
22     >> "2. Sacar de pila \n";
23     >> "3. Imprimir pila \n";
24     >> "4. Salir \n";
25     >> "-----";
26     opcion <<
27     si (opcion == 1) {
28         >> "Ingresa el numero: \n";
29         lectura <<
30         pila.apilar(lectura);
31     } aliter (opcion == 2) {
32         lectura = pila.desapilar();
33         >> "Elemento desapilado: ";
34         >> lectura;
35     } aliter (opcion == 3) {
36         >> pila.toString();
37     } aliter (opcion == 4) {
38         >> "Fin del programa";
39     } finis;
40     >> "Ingresa cualquier tecla para continuar ";
41     lectura <<
42
43 } finis;
44
45 FINIS;

```

Right Pane (Errores AST Tabla de simbolos Pila de procesos Cuartetas Código C):

```

#include <float.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MEMORY_SIZE 1000000
/* Ningun marco llega a ese tamaño: el chequeo del stack deja ese margen. */
#define FRAME_MARGIN 1000
/* Cada llamada del programa también es una llamada de C, con sus temporales en el
stack de C: se detiene antes de agotar los 8 MB que Linux le da por defecto. */
#define C_STACK_LIMIT (7 * 1024 * 1024)

double stack[MEMORY_SIZE]; /* marcos de las funciones; P es la base del actual */
double heap[MEMORY_SIZE]; /* cadenas, arreglos y estructuras; H, la celda libre */
double P;
double H;
uintptr_t stackBase; /* donde empezaba el stack de C al entrar a main */

/* -----
* Nativas: el lenguaje las usa sin declararlas. Una cadena es una
* secuencia de bytes UTF-8 en el heap, terminada en -1.
* ----- */

void checkMemory(void)
{
    char here;
    uintptr_t current = (uintptr_t) &here;
    uintptr_t used = stackBase > current ? stackBase - current : current - stackBase;

    if (H > MEMORY_SIZE || P + FRAME_MARGIN > MEMORY_SIZE || used > C_STACK_LIMIT)
    {
        printf("\nError: memoria agotada (stack o heap).\n");
        exit(1);
    }
}

/* Un numerus desbordado (2147483647 + 1) vale como en Java: sus 32 bits bajos. Pasado
cabe ni en long long (convertirlo sería indefinido en C) y ya no guarda bits exact
ponytall: + - * no se envuelven al calcularse, solo al imprimirse, dividirse o en

```

Bottom Bar: Compilacion exitosa | 42 simbolo(s) | 744 cuarteta(s) | 840 paso(s) de analisis

Código C generado

Abrir carpeta Nuevo archivo Nueva carpeta Abrir Guardar Guardar como Guardar todo Descargar carpeta Compilar Descargar .c Exportar arbol main.pig

▼ pila
▼ utils
utils.y
main.pig
Nodo.z
Pila.z

main.pig X

```
1 // Prueba de la auxiliar con las correcciones que sugiere el...
2 // los "y" faltantes (lineas 20, 33 y 36) y el "FINIS;" final...
3 //
4 // Entrada de ejemplo, un valor por linea: 1 5 x 1 7 x 3 x 2 4
5 // Imprime la pila "7, 5", desapila 7 y luego imprime "5".
6 import Nodo.z
7 import Pila.z
8 import utils.utils.y
9
10 VARIABLES>
11 esto pila : novus Pila();
12 esto opcion : numerus -1;
13 esto lectura : numerus 0;
14
15 MAIOR>
16 imprimirBienvenida();
17
18 dum (opcion != 4) {
19   >> "-----"
20   >> "Ingresar la accion: \n";
21   >> "1. Ingresar en pila \n";
22   >> "2. Sacar de pila \n";
23   >> "3. Imprimir pila \n";
24   >> "4. Salir \n";
25   >> "-----";
26   opcion <<
27   si (opcion == 1) {
28     >> "Ingresar el numero: \n";
29     lectura <<
30     pila.apilar(lectura);
31   } aliter (opcion == 2) {
32     lectura = pila.desapilar();
33     >> "Elemento desapilado: ";
34     >> lectura;
35   } aliter (opcion == 3) {
36     >> pila.toString();
37   } aliter (opcion == 4) {
38     >> "Fin del programa";
39   } finis;
40   >> "Ingresar cualquier tecla para continuar ";
41   lectura <<
42 } finis;
43
44 FINIS;
```

Errores AST Tabla de simbolos Pila de procesos Cuartetas Codigo C

#	Operador	Arg 1	Arg 2	Resultado
0	function			fn_Nodo_Nodo_int
1	+	P	1	t0
2	=[]	stack	t0	t1
3	+	P	2	t2
4	=[]	stack	t2	t3
5	[]=	t1	t3	heap
6	+	P	1	t4
7	=[]	stack	t4	t5
8	+	t5	1	t6
9	[]=	t6	0	heap
10	end			fn_Nodo_Nodo_int
11	function			fn_Nodo_getData
12	+	P	1	t7
13	=[]	stack	t7	t8
14	=[]	heap	t8	t9
15	[]=	P	t9	stack
16	return			
17	end			fn_Nodo_getData
18	function			fn_Nodo_setData_int
19	+	P	1	t10
20	=[]	stack	t10	t11
21	+	P	2	t12
22	=[]	stack	t12	t13
23	[]=	t11	t13	heap
24	end			fn_Nodo_setData_int
25	function			fn_Nodo_getSiguiente
26	+	P	1	t14
27	=[]	stack	t14	t15
28	+	t15	1	t16
29	=[]	heap	t16	t17
30	[]=	P	t17	stack

Compilacion exitosa | 42 simbolo(s) | 744 cuarteta(s) | 840 paso(s) de analisis

Cuartetas

Abrir carpeta Nuevo archivo Nueva carpeta Abrir Guardar Guardar como Guardar todo Descargar carpeta Compilar Descargar .c Exportar arbol main.pig

▼ pila
▼ utils
utils.y
main.pig
Nodo.z
Pila.z

main.pig X

```
1 // Prueba de la auxiliar con las correcciones que sugiere el...
2 // los "y" faltantes (lineas 20, 33 y 36) y el "FINIS;" final...
3 //
4 // Entrada de ejemplo, un valor por linea: 1 5 x 1 7 x 3 x 2 4
5 // Imprime la pila "7, 5", desapila 7 y luego imprime "5".
6 import Nodo.z
7 import Pila.z
8 import utils.utils.y
9
10 VARIABLES>
11 esto pila : novus Pila();
12 esto opcion : numerus -1;
13 esto lectura : numerus 0;
14
15 MAIOR>
16 imprimirBienvenida();
17
18 dum (opcion != 4) {
19   >> "-----"
20   >> "Ingresar la accion: \n";
21   >> "1. Ingresar en pila \n";
22   >> "2. Sacar de pila \n";
23   >> "3. Imprimir pila \n";
24   >> "4. Salir \n";
25   >> "-----";
26   opcion <<
27   si (opcion == 1) {
28     >> "Ingresar el numero: \n";
29     lectura <<
30     pila.apilar(lectura);
31   } aliter (opcion == 2) {
32     lectura = pila.desapilar();
33     >> "Elemento desapilado: ";
34     >> lectura;
35   } aliter (opcion == 3) {
36     >> pila.toString();
37   } aliter (opcion == 4) {
38     >> "Fin del programa";
39   } finis;
40   >> "Ingresar cualquier tecla para continuar ";
41   lectura <<
42 } finis;
43
44 FINIS;
```

PROGRAMA 6:1
import Nodo.z 6:1
import Pila.z 7:1
import utils.utils.y 8:1
▼ esto pila : Pila 11:1
novus Pila() 11:13
▼ esto opcion : numerus 12:1
► - (unario) 12:23
▼ esto lectura : numerus 13:1
0 13:24
▼ INSTRUCCION 16:1
imprimirBienvenida() 16:1
▼ DUM 18:1
► != 18:6
► BLOQUE 18:19

Compilacion exitosa | 42 simbolo(s) | 744 cuarteta(s) | 840 paso(s) de analisis

Árbol AST

Abrir carpeta

Nuevo archivo

Nueva carpeta

Abrir

Guardar

Guardar como

Guardar todo

Descargar carpeta

Compilar

Descargar .c

Exportar arbol

main.pig X

▼ pila

utils

main.pig

Nodo.z

Pila.z

```
1 // Prueba de la auxiliar con las correcciones que sugiere el
2 // los "y" faltantes (lineas 20, 33 y 36) y el "FINIS;" final
3 //
4 // Entrada de ejemplo, un valor por linea: 1 5 x 1 7 x 3 x 2 4
5 // Imprime la pila "7, 5", desapila 7 y luego imprime "5".
6 import Nodo.z
7 import Pila.z
8 import utils.utiles.y
9
10 VARIABLES>
11 esto pila : novus Pila();
12 esto opcion : numerus -1;
13 esto lectura : numerus 0;
14
15 MAIOR>
16 imprimirBienvenida();
17
18 dum (opcion != 4) {
19   >> "-----"
20   >> "Ingresa la accion: \n";
21   >> "1. Ingresar en pila \n";
22   >> "2. Sacar de pila \n";
23   >> "3. Imprimir pila \n";
24   >> "4. Salir \n";
25   >> "-----";
26   opcion <<
27   si (opcion == 1) {
28     >> "Ingresa el numero: \n";
29     lectura <<
30     pila.apilar(lectura);
31   } aliter (opcion == 2) {
32     lectura = pila.desapilar();
33     >> "Elemento desapilado: ";
34     >> lectura;
35   } aliter (opcion == 3) {
36     >> pila.toString();
37   } aliter (opcion == 4) {
38     >> "Fin del programa";
39   } finis;
40   >> "Ingresa cualquier tecla para continuar ";
41   lectura <<
42
43 } finis;
44
45 FINIS;
```

Errores

AST

Tabla de simbolos

Pila de procesos

Cuartetos

Codigo C

Nombre	Categoria	Tipo	Ambito	Linea	Columna
Nodo	Clase	Nodo	global	1	1
Nodo.Nodo	Constructor	void	global	5	5
Nodo.getDato	Metodo	int	global	10	5
Nodo.setDato	Metodo	void	global	14	5
Nodo.getSiguiente	Metodo	Nodo	global	18	5
Nodo.setSiguiente	Metodo	void	global	22	5
Pila	Clase	Pila	global	4	1
Pila.Pila	Constructor	void	global	8	5
Pila.apilar	Metodo	void	global	14	5
Pila.desapilar	Metodo	int	global	22	5
Pila.obtenerCima	Metodo	int	global	30	5
Pila.estaVacia	Metodo	boolean	global	37	5
Pila.obtenerTamano	Metodo	int	global	41	5
Pila.toString	Metodo	String	global	45	5
imprimirBienvenida	Funcion	void	global	2	1
dato	Atributo	int	Nodo	2	5
siguiente	Atributo	Nodo	Nodo	3	5
this	Parametro	Nodo	Nodo	5	5
dato1	Parametro	int	Nodo	5	17
this	Parametro	Nodo	getDato	10	5
this	Parametro	Nodo	setDato	14	5
dato1	Parametro	int	setDato	14	25
this	Parametro	Nodo	getSiguiente	18	5
this	Parametro	Nodo	setSiguiente	22	5
siguiente1	Parametro	Nodo	setSiguiente	22	30
cima	Atributo	Nodo	Pila	5	5
tamano	Atributo	int	Pila	6	5
this	Parametro	Pila	Pila	8	5
this	Parametro	Pila	apilar	14	5
dato	Parametro	int	apilar	14	24
nuevo	Variable	Nodo	apilar	15	9

Compilacion exitosa | 42 simbolo(s) | 744 cuarteta(s) | 840 paso(s) de analisis

Tabla de simbolos

Abrir carpeta

Nuevo archivo

Nueva carpeta

Abrir

Guardar

Guardar como

Guardar todo

Descargar carpeta

Compilar

Descargar .c

Exportar arbol

main.pig

▼ pila

utils

main.pig

Nodo.z

Pila.z

```
1 // Prueba de la auxiliar con las correcciones que sugiere el
2 // los "y" faltantes (lineas 20, 33 y 36) y el "FINIS;" final
3 //
4 // Entrada de ejemplo, un valor por linea: 1 5 x 1 7 x 3 x 2 4
5 // Imprime la pila "7, 5", desapila 7 y luego imprime "5".
6 import Nodo.z
7 import Pila.z
8 import utils.utiles.y
9
10 VARIABLES>
11 esto pila : novus Pila();
12 esto opcion : numerus -1;
13 esto lectura : numerus 0;
14
15 MAIOR>
16 imprimirBienvenida();
17
18 dum (opcion != 4) {
19   >> "-----"
20   >> "Ingresa la accion: \n";
21   >> "1. Ingresar en pila \n";
22   >> "2. Sacar de pila \n";
23   >> "3. Imprimir pila \n";
24   >> "4. Salir \n";
25   >> "-----";
26   opcion <<
27   si (opcion == 1) {
28     >> "Ingresa el numero: \n";
29     lectura <<
30     pila.apilar(lectura);
31   } aliter (opcion == 2) {
32     lectura = pila.desapilar();
33     >> "Elemento desapilado: ";
34     >> lectura;
35   } aliter (opcion == 3) {
36     >> pila.toString();
37   } aliter (opcion == 4) {
38     >> "Fin del programa";
39   } finis;
40   >> "Ingresa cualquier tecla para continuar ";
41   lectura <<
42
43 } finis;
44
45 FINIS;
```

Errores

AST

Tabla de simbolos

Pila de procesos

Cuartetos

Codigo C

< Anterior

Siguiente >

Paso 1 de 840

Pila de llamadas

> programa

Log de procesos

[1] PUSH (regla activa) (6:1) Se activa la regla <programa

Compilacion exitosa | 42 simbolo(s) | 744 cuarteta(s) | 840 paso(s) de analisis

Pila de procesos

EJECUCIÓN C GENERADO

```
gcc salida.c -o salida      # en Windows: wsl -e gcc salida.c -o salida
./salida                    # en Windows: wsl -e ./salida
```

El programa se detiene con un mensaje claro ante *referencia nula* (p. ej. desapilar una pila vacía), *división entera entre cero* (un decimal entre cero da Infinity o NaN), *memoria agotada* o una lectura cuando ya no hay más datos de entrada.

EJEMPLOS

Carpeta / archivo	Qué muestra
pila/	la prueba de la auxiliar ya corregida: objetos .z y una función .y
objetos/	todos los ejemplos del enunciado de Zetariano
matrices/	matrices en PigLatin y Y?, con los literales confirmados por la auxiliar
resistencia.pig, enunciado.pig	PigLatin + Y?: funciones, recursividad, arreglos, estructuras, ciclos

Cada `main.pig` indica en su encabezado la entrada de ejemplo y la salida esperada.